



Module 13: Parallel Algorithms and Multithreading

CPE010 DATA STRUCTURES AND ALGORITHMS

Understanding Key Concepts to Improve Program Efficiency

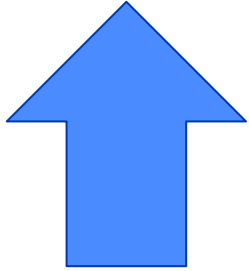
Intended Learning Outcomes

After this module, the student should be able to:

- Create C++ that uses parallel programming for better code performance;
- Create C++ code that implements multithreading and concurrency.

Context

In the modern tech climate, **concurrency** has become an **essential skill for all C++ programmers**.

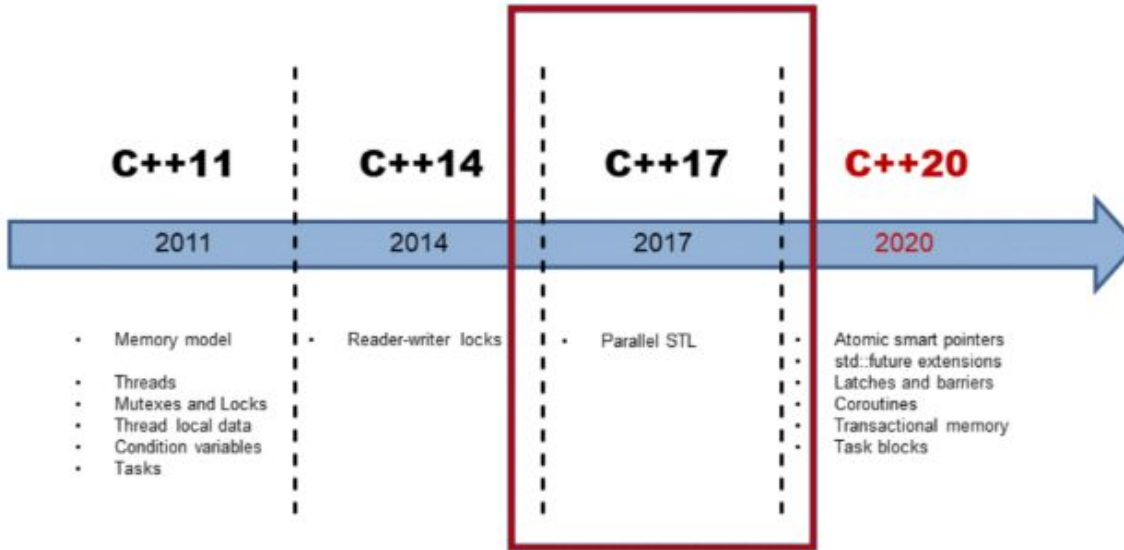


In modern programming, there is an **increase** in:

- **Program Complexity**
- **Computer CPU Cores**

The best way for you to make use of these **multicore machines** is the **coding technique of concurrency**.

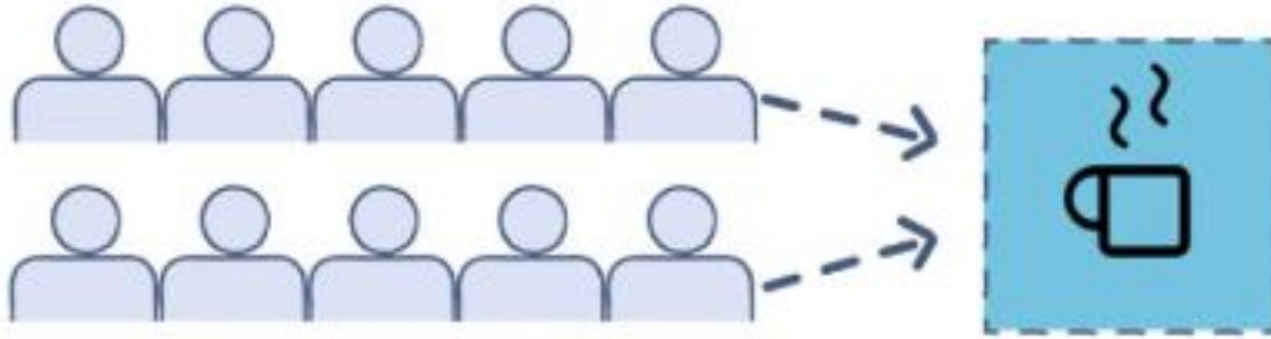
Concurrency and Parallelism with C++



- First introduced in **C++11**.
- Drastically improved/changed in **C++17**.

The addition of parallel algorithms in the Standard Template Library (STL) greatly improved concurrent code.

Concurrency Defined

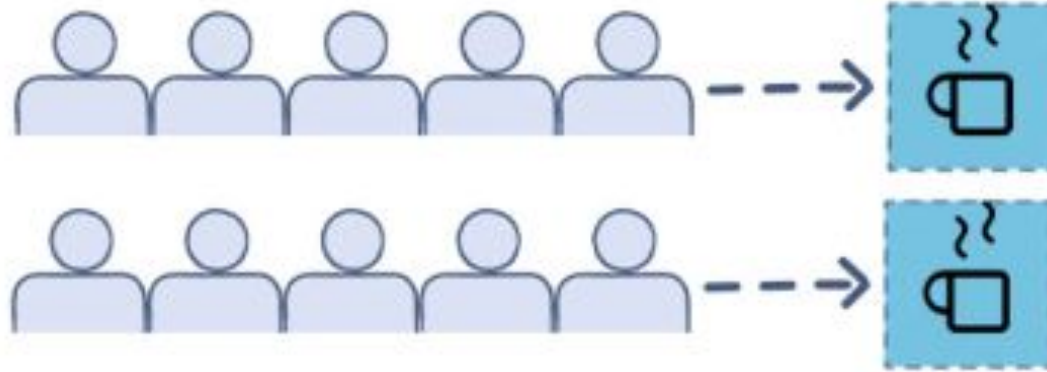


Two Queues & Single Espresso Machine

Concurrency happens when **two tasks overlap**.

- Has **data race** issues.
- Data race occurs when at least two *threads* can simultaneously access a variable or memory, and at least one of them want to access the variable.

Parallelism Defined



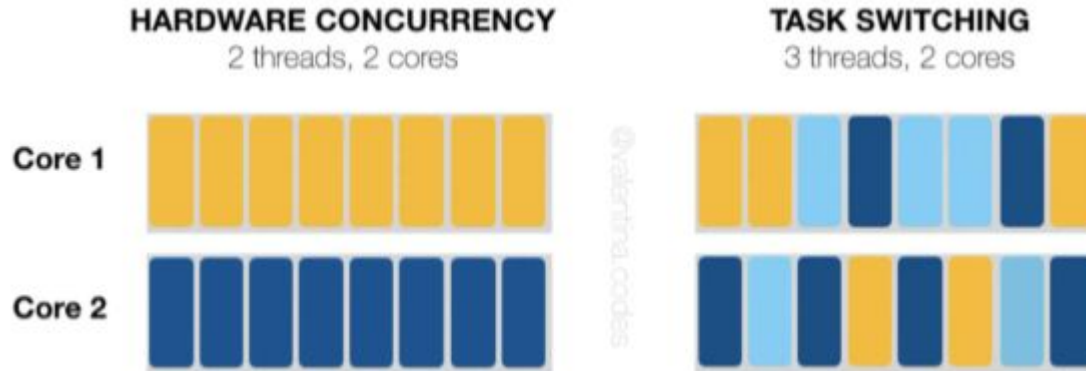
Two Queues & Two Espresso Machines

In Parallelism, multiple copies of the program run **simultaneously** but on **different** data.

- Could be used to send requests to different websites but give each copy of the program with different URLs.
- No communication; only parallel execution.

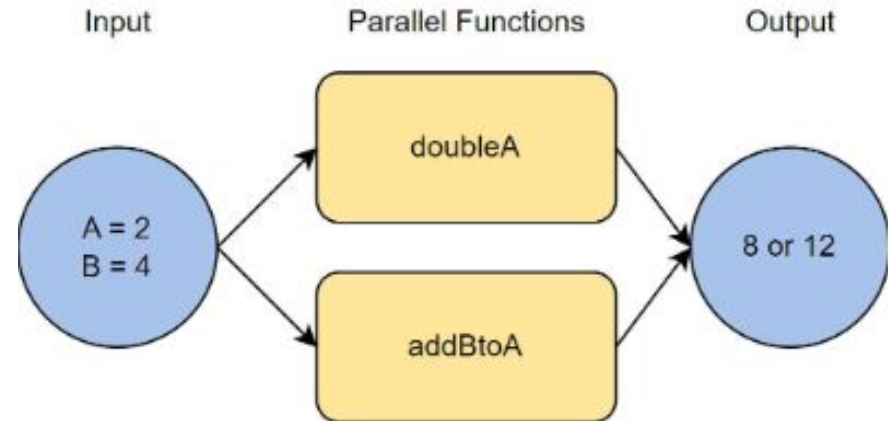
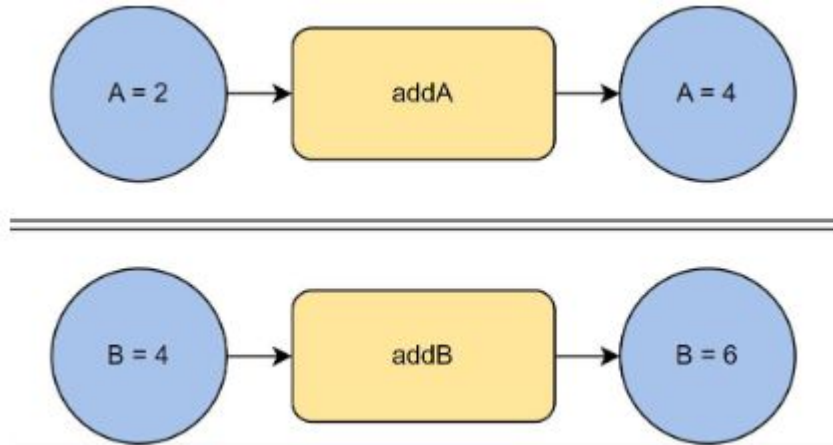
C++ Multithreading

- We use **multithreading** to implement concurrency.
- Other languages also have methods for multithreading, **however, C++ stands-out because of the lower-than-average overhead cost.**
- Not advisable for short simpler functions.



Parallel Algorithms / Functions

- Parallel functions can **significantly speed-up** the operations.
 - They automatically use more CPU resources.
- Parallel functions are best used for functions without **dependencies** or **data editing**.



*Is there data dependency or data editing?
THERE IS BOTH.*

Concept Check:

Explain the difference between parallelism and concurrency.



Implementing

Multithreading in C++

Using <thread>

- Use `std::thread` to create a new thread object and pass a **callable**.
 - A callable is an executable code that we want to execute when the thread is running.
- Callables can be defined in 3 ways:
 - Using function object;
 - Using function pointer;
 - Using a Lambda expression.

Using the Function Object

To use the function object as the callable:

- Create a class
- Overload the () operator.
- Include code to execute inside () once thread object is created.

```
// Define the class for function object
class functionObject_class {
// Overload () operator
void operator() (params)
{
    // code to be executed
}
};
// Create thread object
std::thread thread_object(functionObject_class () , params)
```

First parameter is overloaded function then parameters.

Using Function Pointer

Defined by:

```
void funct_call(params)
    //code to be executed
}
```

Creating thread object by:

Pass the name of the function as first parameter.

```
std::thread thread_obj(funct_call, params);
```

Parameters for the function are passed as second parameter to the thread object.

Using a Lambda Function

The callable Lambda expression can be defined as:

```
// Define a lambda expression
auto f = [](params) {
    // code for execution
};
```

Thread object created by:

First parameter is the expression.

```
std::thread thread_object(f, params);
```

Second are the parameters of the expression.

std::thread join function

In cases where we want to finish the current thread before the next action, we use the join method.

Example:

```
int main()
{
    std::thread t1(callable_code);
    ...
    t1.join();
    ...
}
```

The main function will have to wait to continue until `thread t1` finishes. The join function of thread blocks other actions/functionality until the thread calling finishes its execution.

Seatwork

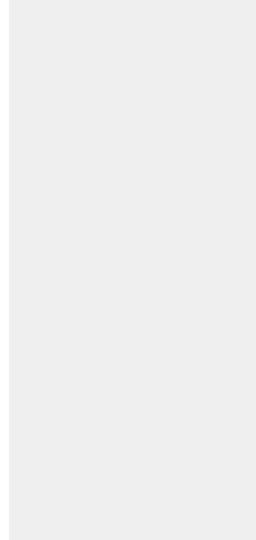
Create code to demonstrate multithreading in C++

The following must be satisfied:

- Take 2 parameters i and j.
- Function must display counting from i -> j.

Steps:

- Implement **all three** callables.
- Create thread objects and pass each callable.
- Wait for every thread to finish.



Sample Answer

- Implement **all three** callables.

```
int main(){  
    //callable Lambda expression  
    auto f : void (int, int) const = [](int i, int j) -> void {  
        std::cout << "Callable Lambda Expression\n";  
        for(int k = i; k <= j; k++){  
            std::cout << k << " ";  
        }  
        std::cout << "End of Lambda Expression\n";  
    };  
};
```

```
//callable function  
void FunctionCall(int i, int j){  
    std::cout << "Callable Function Display\n";  
    for(int k = i; k <= j; k++){  
        std::cout << k << " ";  
    }  
    std::cout << "End of FunctionCall\n";  
};
```

```
//callable object  
class ObjectCall{  
public:  
    void operator()(int i, int j){  
        std::cout << "Callable Object Display\n";  
        for(int k = i; k <= j; k++){  
            std::cout << k << " ";  
        }  
        std::cout << "End of ObjectCall\n";  
    }  
};
```

- Create **thread objects** and pass each callable.

```
std::thread th1( &: FunctionCall, 2, 4);  
std::thread th2( f: ObjectCall(), 2, 4);  
std::thread th3( &: f, 2, 4);
```

- **Wait** for every thread to finish.

```
th1.join();  
th2.join();  
th3.join();
```

- **Output:**

Callable Function Display

Callable Object Display

2 3 4 End of ObjectCall

2 3 4 End of FunctionCall

Callable Lambda Expression

2 3 4 End of Lambda Expression

Process finished with exit code 0

Concept Check:

Modify the sample code so that instead of counting; it returns the sum of the values between i and j .



Thanks for participating!

Submit your questions through the discussion
Q&A found on module 12.